

SIMATS ENGINEERING ASSESSMENT TOOL 3

COURSE CODE: CSA1402

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO4: Examine intermediate code generation techniques to enhance code optimization (BL4)

Assessment Tool Weightage: 10%

QUESTIONS:

Total Marks:50

Annexure A

DATA INTERPRETATION

Code of Conduct:

I _Monika.R(192324281)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Monika.R

CSA1402- COMPILER DESIGN

CO4 - AT3

DATA INTERPRETATION

NAME: MONIKA R

REG.NO: 192324281

Submitted To:

Dr. N. Deepa

Submitted on:

22-08-2026

Annexure A

DATA INTERPRETATION

Q1. Intermediate Code Analysis and Optimization

Given the three-address code:

$t1 = x + y$ $t2 = t1 * z$ $t3 = t2 / w$ $t4 = t3 - p$

Analyze the sequence of operations and construct the equivalent expression tree. Evaluate how temporary variables improve intermediate code generation and reduce repeated computations.

Q2. Symbol Table and Memory Allocation

Given the following symbol table entries:

Variable Data Type Size (Bytes)

p char 1

q double 8

r int 4

s float 4

Interpret how the compiler allocates memory for these variables and calculate the total memory required. Analyze how alignment and data types affect memory organization.

Q3. Optimization in Intermediate Code

Given the intermediate code:

$t1 = m * n$ $t2 = p + q$ $t3 = m * n$ $t4 = t2 - t3$

Trace the execution step-by-step and identify optimization opportunities such as common subexpression elimination and redundant computation removal. Explain how optimization improves execution efficiency.

Q4. Control Flow and Boolean Expression Handling

Given the Boolean expression:

if ($x > y \parallel p < q$) Intermediate code:

if $x > y$ goto L1 if $p < q$ goto L1 goto L2

L1: ...

Interpret the control flow of the given intermediate representation and explain how short-circuit evaluation minimizes unnecessary condition checking during execution.

Q5. Backpatching and Flow Control

Given:

Instruction Target

200: if $x == y$ goto _

201: goto _

Backpatch lists:

- truelist = {200}
- falselist = {201}

Analyze the role of backpatching in intermediate code generation. Demonstrate how target addresses are updated during control flow handling and explain its importance in Boolean expression translation.

ANSWERS

Q1. INTERMEDIATE CODE ANALYSIS AND OPTIMIZATION

Given Three-Address Code

$$t1 = x + y$$

$$t2 = t1 * z$$

$$t3 = t2 / w$$

$$t4 = t3 - p$$

Analysis of the Sequence of Operations

The operations are performed sequentially as follows:

1. $t1 = x + y$

2. $t2 = t1 * z$

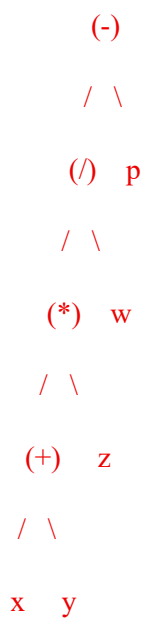
3. $t3 = t2 / w$

4. $t4 = t3 - p$

Therefore, the final expression represented by the intermediate code is:

$$t4 = ((x + y) \times z) / w - p$$

Equivalent Expression Tree



The expression tree represents:

$$((x + y) \times z) / w - p$$

Temporary Variables

The temporary variables are used to store intermediate results:

$$t1 \rightarrow x + y$$

$$t2 \rightarrow t1 \times z$$

$$t3 \rightarrow t2 / w$$

$$t4 \rightarrow t3 - p$$

A compiler can reduce the use of temporary variables by using CPU registers.

For example:

$$R1 = x + y$$

$$R1 = R1 * z$$

$$R1 = R1 / w$$

$$R1 = R1 - p$$

This reduces memory accesses and improves execution efficiency.

Optimization

There is no repeated subexpression in the given code. Therefore, common subexpression elimination is not applicable.

However, temporary variables can be reduced by register allocation and temporary reuse.

Conclusion

The equivalent expression is:

$$((x + y) \times z) / w - p$$

The compiler can improve efficiency by keeping intermediate values in registers instead of repeatedly storing them in memory.

Q2. SYMBOL TABLE AND MEMORY ALLOCATION

Given Symbol Table

Variable	Data Type	Size (Bytes)
p	char	1
q	double	8
r	int	4
s	float	4

Memory Allocation

The variables require the following storage:

p → 1 byte

q → 8 bytes

r → 4 bytes

s → 4 bytes

Therefore, the total data size without considering alignment is: $1 + 8 + 4 + 4$

= 17 bytes

Hence:

Total raw memory requirement = 17 bytes

Effect of Alignment and Padding

Modern computer systems often require variables to be stored at properly aligned memory addresses.

For example, if a double requires 8-byte alignment, padding may be inserted before it.

Assuming allocation starts at address 1000:

Address	Variable
1000	p (1 byte)
1001–1007	Padding (7 bytes)
1008	q (8 bytes)
1016	r (4 bytes)

1020 s (4 bytes)

Thus, the total memory occupied in this example is: **24 bytes**

The actual amount of padding depends on the processor architecture, compiler, ABI, and starting address.

Effect of Data Types

Different data types require different amounts of memory:

char → 1 byte

int → 4 bytes

float → 4 bytes

double → 8 bytes

The compiler stores information about each variable in the **symbol table**.

Typical symbol-table information includes:

- Variable name
- Data type
- Size
- Scope
- Memory address or offset

Conclusion

The raw memory requirement for the variables is:

17 bytes.

However, the actual memory allocated by the compiler may be greater because of **alignment and padding**.

Q3. OPTIMIZATION IN INTERMEDIATE CODE

Given Intermediate Code

$t1 = m * n$

$t2 = p + q$

$t3 = m * n$

$t4 = t2 - t3$

Step-by-Step Execution

Step 1 $t1 = m * n$

The multiplication of m and n is performed and the result is stored in $t1$.

Step 2 $t2 = p + q$

The addition of p and q is performed and the result is stored in $t2$.

Step 3 $t3 = m * n$

The compiler calculates $m * n$ again.

This is unnecessary because the same expression was already calculated in Step 1.

Step 4 $t4 = t2 - t3$

The value of $t3$ is subtracted from $t2$.

Optimization Opportunity

The expression: $m \times n$

occurs twice:

$t1 = m * n$

$t3 = m * n$

This is a **common subexpression**.

The compiler can eliminate the second calculation and reuse $t1$.

Optimized Intermediate Code

$t1 = m * n$

$t2 = p + q$

$t4 = t2 - t1$

Comparison

Original code:

2 multiplications

1 addition

1 subtraction

Optimized code:

1 multiplication

1 addition

1 subtraction

Thus, one unnecessary multiplication is eliminated.

Optimization Technique

This optimization is called:

Common Subexpression Elimination (CSE)

Benefits

Optimization improves execution efficiency by:

1. Reducing the number of operations.
2. Reducing execution time.
3. Reducing unnecessary use of temporary variables.
4. Reducing register and memory requirements.
5. Improving overall program performance.

Conclusion

The optimized code is:

$t1 = m * n$

$t2 = p + q$

$t4 = t2 - t1$

This avoids recomputing the common subexpression $m * n$.

Q4. CONTROL FLOW AND BOOLEAN EXPRESSION HANDLING

Given Boolean Expression

if ($x > y \parallel p < q$)

Intermediate Code

if $x > y$ goto L1

if $p < q$ goto L1

goto L2

L1: ...

Control Flow

The first instruction checks whether:

$x > y$

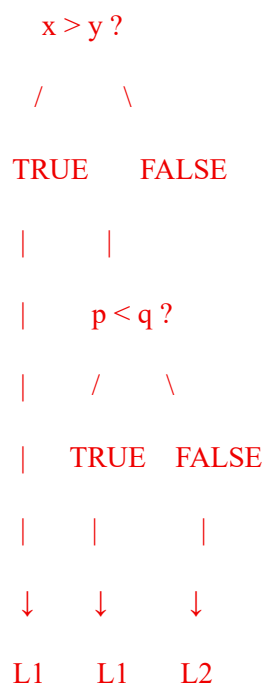
If the condition is true, control immediately jumps to L1.

If it is false, the second condition is checked: $p < q$

If $p < q$ is true, control jumps to L1.

If $p < q$ is also false, control goes to L2.

Control-Flow Representation



Short-Circuit Evaluation

The Boolean operator used is **OR** (**||**).

The expression is:

(x > y) OR (p < q)

For an OR operation, if the first condition is true, the second condition does not need to be evaluated.

Therefore:

if x > y goto L1

If x > y is true, the program directly goes to L1.

The condition p < q is evaluated only when x > y is false.

Advantages of Short-Circuit Evaluation

Short-circuit evaluation:

1. Avoids unnecessary condition checking.
2. Reduces execution time.
3. Improves program efficiency.
4. Avoids evaluating unnecessary expressions.
5. Is useful when expressions contain expensive computations.

Meaning of Labels

L1 → Boolean expression is TRUE

L2 → Boolean expression is FALSE

Therefore, the intermediate code correctly implements: **if (x > y || p < q)**

using short-circuit evaluation.

Q5. BACKPATCHING AND FLOW CONTROL

Given

Instruction	Target
200: if x == y goto _	—
201: goto _	—

Backpatch Lists

truelist = {200}

falselist = {201}

Definition of Backpatching

Backpatching is a technique used during intermediate-code generation in which the compiler temporarily leaves the target address of a jump instruction blank.

When the actual target address becomes known, the compiler fills in the missing address.

Backpatching is mainly used for:

- Boolean expressions
- Conditional statements
- if-else statements
- while loops
- for loops
- Short-circuit evaluation

Interpretation of Instruction 200

200: if x == y goto _

If the condition:

x == y

is true, control must jump to the true target.

Therefore: truelist = {200}

Interpretation of Instruction 201

201: goto _

This represents the false path.

Therefore: falselist = {201}

Backpatching Process

Suppose the compiler later determines:

True target = 205

False target = 210

The compiler updates the instructions as follows:

200: if x == y goto 205

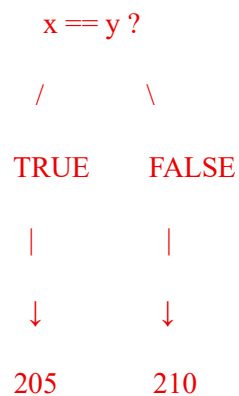
201: goto 210

Thus:

truelist = {200} → 205

falselist = {201} → 210

Control Flow



Role of Backpatching in Boolean Translation

During the initial generation of intermediate code, the compiler may not know the exact addresses of future instructions.

For example:

if (x == y)

 statement1;

else

 statement2;

The compiler can initially generate:

200: if x == y goto _

201: goto _

After generating the code for statement1 and statement2, their addresses become known.

The compiler then performs backpatching.

Importance of Backpatching

Backpatching:

1. Allows forward jumps to be generated before their destinations are known.
2. Simplifies intermediate-code generation.
3. Supports Boolean expression translation.
4. Supports short-circuit evaluation.
5. Helps generate efficient control-flow code.
6. Is useful in one-pass or syntax-directed intermediate-code generation.

Basic Backpatching Process

Generate jump instruction



Store instruction number



Create truelist/falselist



Determine target address



Backpatch the target address

Final Conclusion

Backpatching is an important compiler technique for filling in incomplete jump targets after their destination addresses become known.

The key concept is: **Generate → Store → Determine Target → Backpatch**

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand